Non-Functional Requirements

CS1040 Program Construction

Eng Prof Chandana Gamage

BSc Eng Hons, MEng, PhD, CEng

Department of Computer Science & Engineering

University of Moratuwa

January 26, 2026

What's in a name?

"What's in a name? That which we call a rose by any other name would smell just as sweet."

William Shakespeare uses this line in his play *Romeo and Juliet* to convey that the naming of things is irrelevant. Do you agree?

- Why call *Program Construction?*
- Why not *Programming?*

Program Construction Methodology

Program construction methodology has many different phases of activity.

- One part of it covers the overall system including hardware, software, users, operating environment, rules and regulations, etc.
- Other part covers software development.

- An **algorithm** can be represented using a natural language, flow charts, pseudo-code, or a structured language.
- Programming is coding an algorithm to a **program** using a programming language like C, C++, C#, Java, Python, Go, JavaScript, Go, ...
- Program construction is a **methodological process** to start from a user's requirements and end with a working program as the final product.

Program Construction Methodology - System Level

Example: The customer needs a system to collect rainfall data island-wide in real-time at a low-cost using crowd-sourcing.

- System requirements analysis
- System requirements specification
- System architecture design
- System validation (checking if the specification captures the customer's requirements)

Program Construction Methodology - Software Level

Example: Record the start and end of a rainfall event, Provide a subjective evaluation of amount of rainfall, Transmit data to a central repository, etc.

- Software requirements analysis
- Software requirements specification
- Software architecture design
- Software component/module design
- Program coding and testing
- Software deployment and maintenance
- Software verification (checking if the software meets the specifications)

A Classification of Requirements

Requirements form the foundation of program construction: the more accurately the requirements are elicited, recorded, and used; the more satisfying the final outcome from the users point of view.

Requirements are broadly classified to two types.

1. Functional requirements
2. Non-functional requirements

What are Functional Requirements?

Two broad approaches to define functional requirements:

1. Based on **functionality**
2. Based on **behaviour**

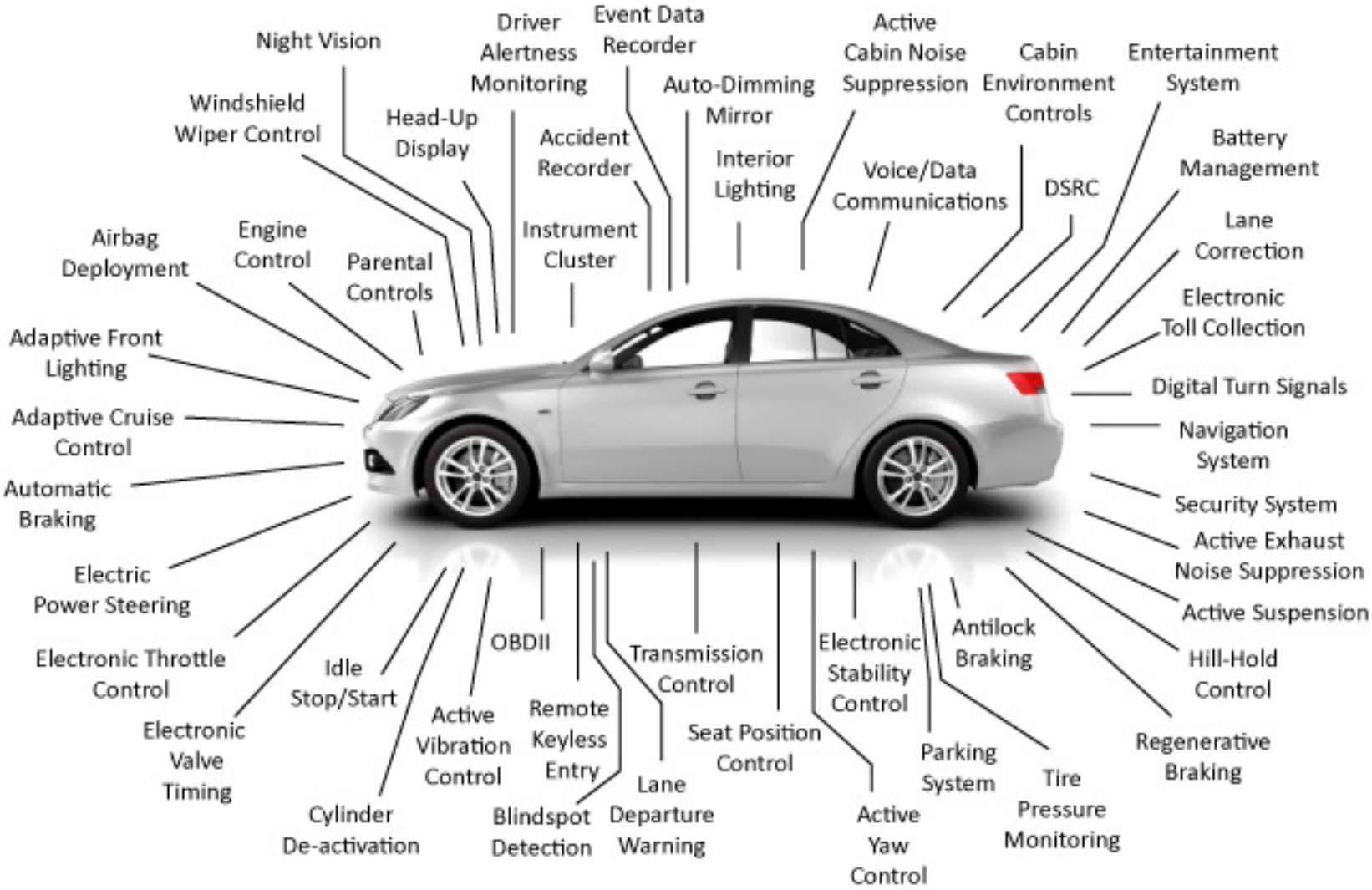
- A **functional** requirement specifies:
  - a function that a system must be able to perform, or
  - what the product must do, or
  - what the system must do
- A **behavioural** requirement specification describes:
  - the behavioural aspects of a system or
  - the behavioural relationship between input/stimuli and output/response in a system

A Definition for Functional Requirements

A generally agreed definition of functional requirements is that it is *a statement of a piece of required functionality or a behavior that a system will exhibit under specific conditions*

[Grady Booch, James Rumbaugh, and Ivar Jacobson]

Complexity of Systems



Each feature/function of this smart car is provided using embedded computing (a combination of SW and HW) that has associated non-functional properties

What are Non-Functional Requirements?

Conventionally, non-functional requirements are identified as **illities**

- Definition of non-functional requirements are generally based on *soft* criteria, which tend to be:
  - subjective,
  - ambiguous, and
  - conflicting

Example: a POV video stream processing software module for a drone that needs to be both **fast** (for real-time operation) and **power efficient** (to conserve battery power and extend operational range).

-ility/ilities

- understandability
- usability
- modifiability
- inter-operability
- reliability
- portability
- dependability
- flexibility
- availability
- maintainability
- scalability
- (re-)configurability,
- customizability
- adaptability
- stability
- variability
- volatility
- traceability
- verifiability
- predictability
- compatibility
- survivability
- accessibility
- ...

- ity/ities

- security
- simplicity
- clarity
- ubiquity
- integrity
- safety
- modularity
- ...

-ness

- user-friendliness
- robustness
- timeliness
- responsiveness
- correctness
- completeness
- cohesiveness
- ...

... and many more

- convenience
- comfort
- performance
- efficiency
- accuracy
- precision
- aesthetics
- consistency
- coherence
- fault tolerance
- self-healing capability
- autonomy
- cost
- development time
- time-to-market
- long-lasting battery
- low coupling
- ...

Example: Performance 1/2

Performance is a critical requirement for both interactive and non-interactive systems.

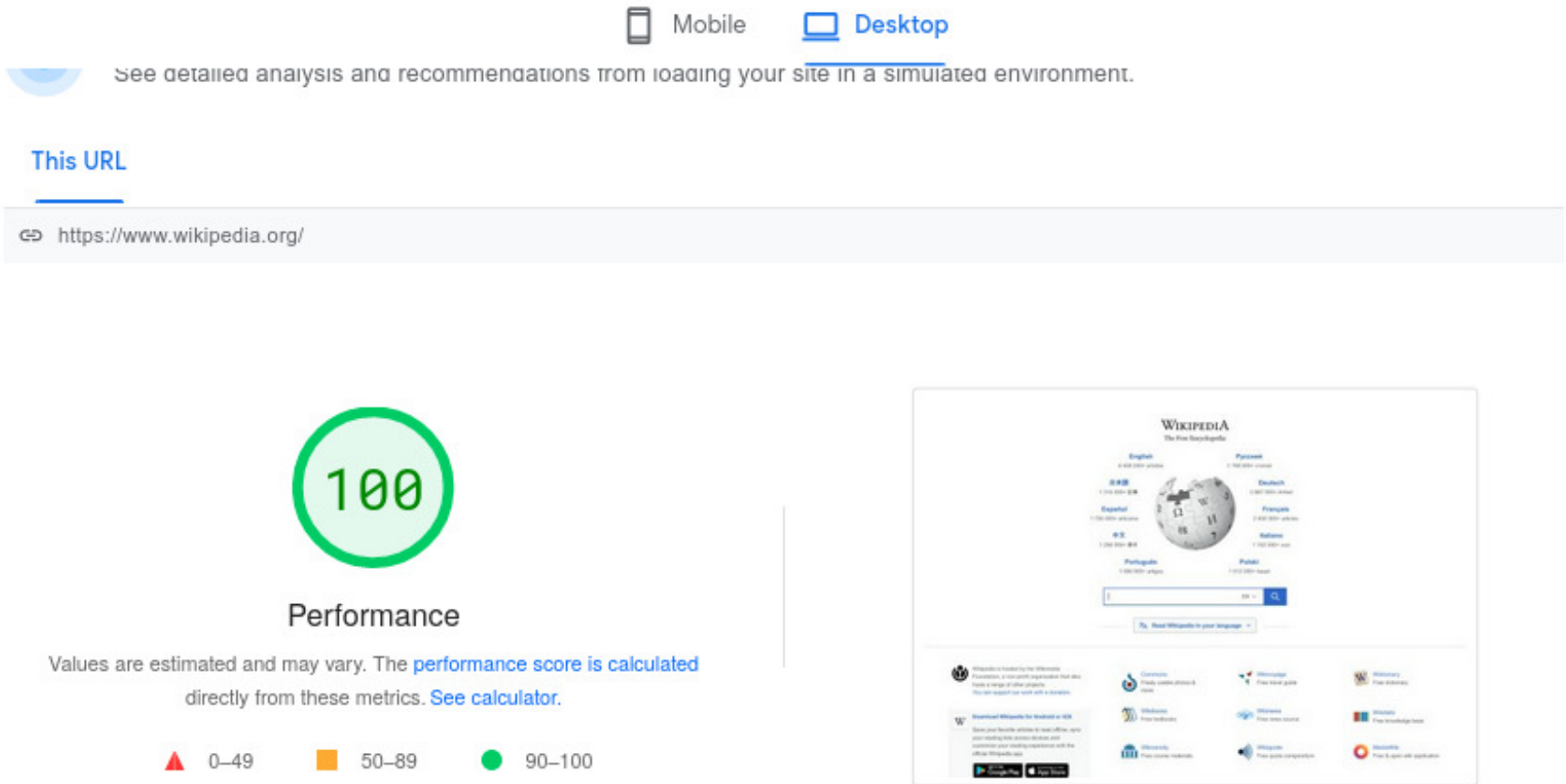
- In an **interactive** software system, performance defines how fast it responds to certain user actions under a certain workload
  - For example, the loading of a web page when the URL is clicked or typed or screen redraw in a computer game
- In a **non-interactive** software system, performance is measured by the amount of time taken to complete a certain task under a certain workload
  - For example, the training of a ML model using a particular training data set or sorting of a collection of data records

Example: Performance 2/2

Performance is impacted by a range of factors, many of them outside the control of engineers who do program construction.

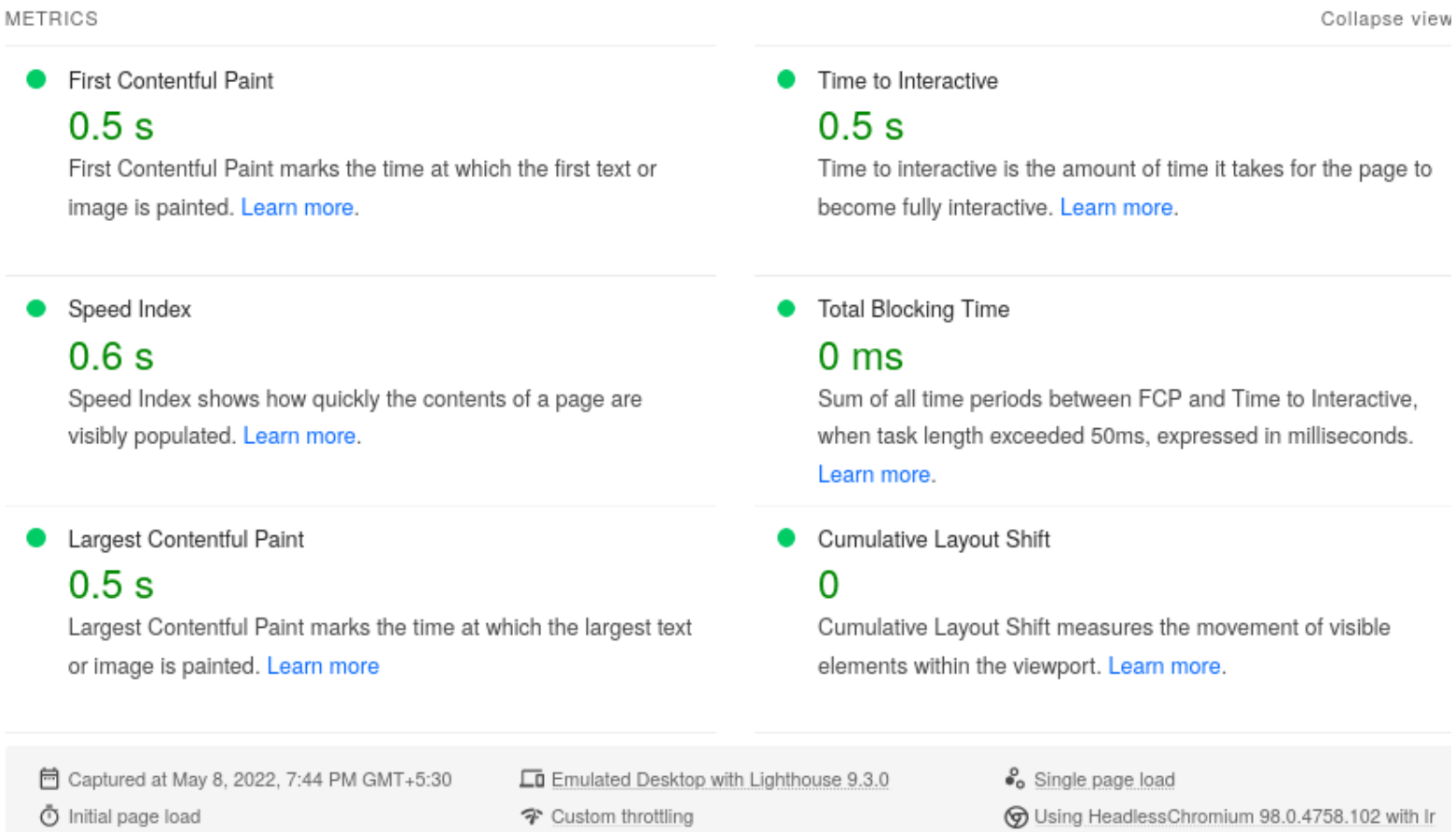
- Generally, the performance metric specifies how much time a user must wait before the target operation is completed (the rendering of web page, rendering of gaming scenery, processing of a transaction, notification of task completion in ML model training or data sorting, etc) in the **context** of *execution environment* such as
  - Hardware resources (processors, processor cores, memory, storage, special hardware, etc)
  - System resources (OS type, libraries, and system parameters defining resource availability and limitations)
  - Run-time environment (for example, the number of users simultaneously active at the moment, background tasks in execution, etc)

Example: PageSpeed Insights 1/2



Google's page speed insights is a tool to measure a set of different performance score considering multiple scenarios such as type of network connection, type of content, and type of platform (desktop or mobile)

Example: PageSpeed Insights 2/2



Performance and Usability

Jakob Nielsen [1], a Danish web usability consultant and human-computer interaction researcher considered the *guru of Web page usability* has suggested 3 main metrics for response time.

- **0.1** second - the limit after which the system reaction does not seem instantaneous
- **1** second - when user will notice the delay, but without interrupted flow of thought
- **10 seconds** - when user attention is completely lost

Although these metrics were suggested nearly 30 years ago in 1993 [2], as they are based on how human interaction works and determined by human perceptual abilities, the metrics remain meaningful and valid.

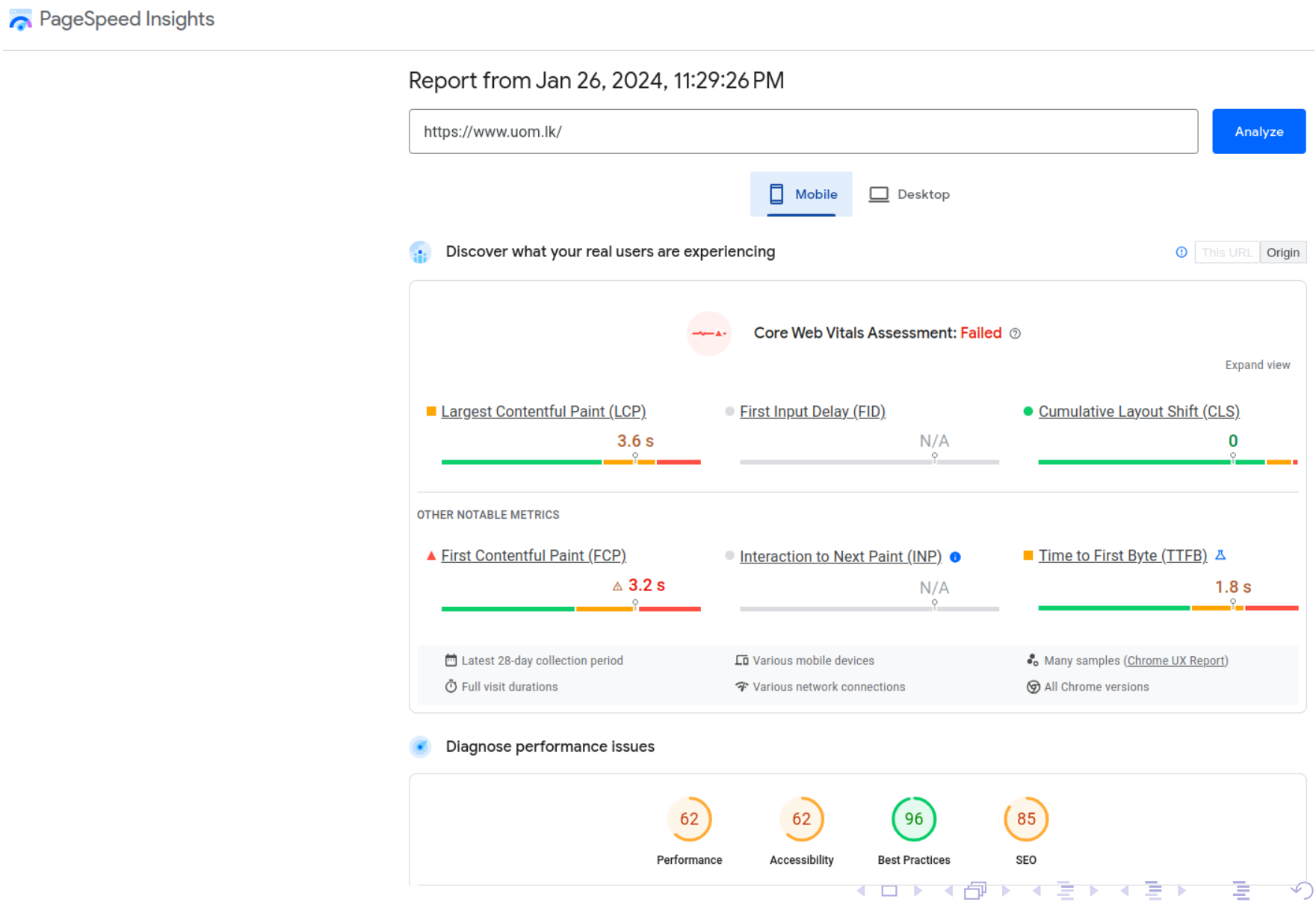
1. `en.wikipedia.org/wiki/Jakob_Nielsen_(usability_consultant)`
2. `www.nngroup.com/articles/response-times-3-important-limits/`

Example: Security

This is a complex non-functional requirement that seek to protect data and services/functions from unauthorized access and malware attacks.

- In program construction, most of the security non-functional requirements are mapped to specific functional requirements.
- Some example mappings are:
  - **identification and authentication** is mapped to a **login** process.
  - **authorized access** is mapped to an **access control matrix** of user identities, objects and system resources, and access rights.
  - **confidentiality** is mapped to an **encryption** process.

An example to try



Summary

Requirements must be documented to be easily understood by non-technical readers (customers) and technical readers (software developers).

	Functional requirements	Non-functional requirements
Objective	Describe **what** product does	Describe **how** product works
End result	Define product **features**	Define product **properties**
Focus	On user **requirements**	On user **expectations**
Documentation	Captured in Use Cases	Captured as a quality attribute
Origin type	Defined by user (mostly)	Defined by developers/experts
Testing	Component, Module, API, UI, ...	Performance, Usability, Security, ...